Student application 2

Linear regression and optimization

Problem statement

A service team wants to estimate how many milliseconds a new request will take, before it runs. This time the required output is a numerical duration, not a slow/not-slow label. Investigate whether payload size and the number of waiting jobs help predict that duration.

What you are given. The notebook generates 600 fictional past requests, each with payload in MB, queue length, and measured duration in ms. It includes random variation, so even a useful model will make errors. Supplied partitions contain 360 training, 120 validation, and 120 test rows. The generator and split are already written; no dataset collection is required.

Concrete example. For an arriving request carrying 4 MB with 3 jobs waiting, the model should return one number in ms. The simulator has a mean duration of 40 + 8*4 + 3*3 = 81 ms for these inputs; a particular observed duration may differ because of noise. The exercise is to estimate a rule from past observations, not to copy the simulator coefficients into a predictor.

The question to answer. How much does a fitted two-input linear rule improve on always predicting the average training duration, and how does one gradient update change its coefficients?

Your assignment

1. Fit a constant training-mean baseline and a linear model using both input columns; compare validation errors in ms.
2. Complete one gradient update on the supplied scaled design matrix and verify that it lowers training error. This small calculation illustrates fitting; it does not replace the fitted model from task 1.
3. Evaluate the fixed two-input linear model and baseline on test rows; interpret the fitted slopes and remaining errors.

Supplied versus your work. Data generation, partitions, the gradient-step setup, and score reports are supplied. You fill three task cells; the two-input model family and step size are fixed for this application.

When you are finished. Submit both error comparisons, the three updated coefficients from the gradient illustration, test predictions, and explanations of slope units and residual noise. Preserve the declared evaluation protocol; do not tune a new model after seeing test errors.

Preparation and notation

Preparation. Allow 60–90 minutes after the lecture. Imports, data, and checks are supplied; no data download or GPU is required. Open the matching notebook:

`02_linear_regression_and_optimization_student.ipynb`.

How to work. Run imports and data first. For each task, read its goal, inputs, steps, command explanations, and expected result. Replace each `...` in the starter and remove the intentional `raise NotImplementedError(...)` line. Run the following check. Correct your code if it fails; keep checks, partitions, and seeds unchanged.

Notation. `tr` means training, `va` validation, and `te` test. `X` denotes raw features; `Z` denotes prepared numeric features. A shape `(rows, columns)` describes a matrix; `(rows,)` describes a vector. Class labels are 0 or 1; probabilities range from 0 to 1.

Variable dictionary

Name	Meaning
`X`	600 rows with payload (MB) and queue (number of waiting requests).
`y`	Measured latency in ms: 40 + 8*payload + 3*queue + random noise with SD 8 ms.
`Xtr, Xva, Xte`	Raw feature tables with 360 training, 120 validation, and 120 test rows; each has two columns.
`ytr, yva, yte`	Matching observed latencies for the three tables. These are real numbers, not classes.
`mean_baseline, linear`	The two model objects you will fit: a constant training-mean predictor and a linear predictor.
`D, target`	Supplied before task 2: D is a (360, 3) design matrix of ones and two scaled training features; target is the length-360 training latency array.
`beta, g, beta_next`	Length-3 coefficient vectors ordered intercept, scaled payload slope, scaled queue slope. beta starts at zero; g is the gradient; beta_next is one update.
`prediction`	The length-120 array of final test predictions you create in task 3.

Keep the experiment valid

Creating a model sets its options. `fit` learns and stores state; `predict`, `predict_proba`, and `transform` apply that state. Training labels belong in fitting; validation and test labels belong in scoring. Record fixed or validation-selected choices before inspecting test. Application 1 uses validation only. Reusing the service test set across these classroom applications is not independent new confirmation.

Task 1. Fit two predictors

Goal. Create two fitted prediction rules and inspect their validation errors.

Inputs. Use Xtr and ytr for fitting. The evaluation cell already supplies Xva and yva.

1. Create mean_baseline using DummyRegressor(strategy="mean"); fit it on Xtr, ytr.
2. Create linear using LinearRegression(); fit it on the same training rows.
3. Run the supplied validation table. Identify which row is the baseline and record MAE and RMSE in ms.
4. The two-feature linear family is specified in advance for this exercise. Do not add features or choose settings from its test error.

Commands and their purpose

`DummyRegressor(strategy="mean")`

Creates a model that learns the mean of the training targets and predicts that same number for every row.

`LinearRegression()`

Creates an ordinary least-squares model with an intercept. The input columns determine which slopes it fits.

`model.fit(Xtr, ytr)`

Estimates model parameters using only training examples and stores them in model.

`model.predict(Xva)`

Applies fitted coefficients to validation inputs and returns one predicted latency per row.

`regression_scores(actual, predicted)`

Supplied helper returning MAE_ms and RMSE_ms. MAE averages absolute errors; RMSE takes the square root of average squared errors. Both are in ms; lower is better.

Starter code

```
mean_baseline = DummyRegressor(strategy="mean")
mean_baseline.fit(..., ...)
linear = LinearRegression()
linear.fit(..., ...)
```

Expected result. Two fitted model objects named mean_baseline and linear, followed by a two-row validation error table.

If you get stuck. Do not pass yva to fit. A successful fit is not a claim of zero error: the generated outcomes include noise.

Task 2. Complete one gradient step

Goal. Translate the half-MSE gradient into array operations and take one descent step.

Inputs. The supplied cell creates D, target, and beta. D includes the intercept column, and its two feature columns use training-based scaling.

1. Calculate prediction-minus-target errors as D @ beta - target. Its shape is (360,).
2. Calculate g as D.T @ error divided by len(target). This is the gradient of half MSE, not full MSE.
3. Set beta_next = beta - 0.1*g. Do not overwrite beta before the supplied before/after comparison.
4. Run the check: the gradient and new coefficients have length three, and one update reduces this training loss. Explain why one step is not the final least-squares fit.

Commands and their purpose

`D @ beta`

Matrix-vector multiplication: produces one prediction per design row.

`D.T`

Transposes D from shape (360, 3) to (3, 360), so feature-weighted errors sum into three gradient components.

`len(target)`

Number of training observations; dividing the summed gradient by it forms an average.

`np.column_stack([ones, features])`

In the supplied setup, places a column of ones beside the scaled features. np.ones creates the ones; to_numpy converts the target Series into an array, and np.zeros initializes the coefficient vector.

`StandardScaler().fit(Xtr)`

In the supplied setup, learns training means and scales. Its transform method reuses them to construct D.

`np.mean(values ** 2)`

Squares each value and averages the results; used by the supplied loss comparison.

Starter code

```
error = D @ beta - target
g = ...
beta_next = ...
```

Expected result. g and beta_next each have shape (3,). The new training squared error is smaller for this supplied step and scaled design.

If you get stuck. Use @, not elementwise *, for the two matrix-vector products. The formula uses prediction-minus-target; reversing that sign makes the stated update go uphill.

Task 3. Evaluate the fixed linear model

Goal. Evaluate the already fitted linear model once on the final test partition.

Inputs. Use the fitted linear and mean_baseline objects from task 1, plus Xte and yte. The task-2 beta_next is only a one-step illustration and is not the final model.

1. Call linear.predict(Xte) and store the result in prediction.
2. Run the supplied test table comparing the baseline and linear model.
3. Read linear.coef_ in the same order as X columns: payload then queue. Explain both slopes with units.
4. Record the frozen model and both test errors. Explain why even the correct model family cannot predict the independent noise exactly.

Commands and their purpose

`linear.predict(Xte)`

Returns fitted-model predictions for the 120 held-out test feature rows; it does not fit again.

`linear.coef_`

A learned attribute containing the two slopes. The trailing underscore marks a fitted attribute in scikit-learn.

`mean_baseline.predict(Xte)`

Returns the training target mean for each test row, making a comparable reference on exactly the same rows.

Starter code

```
prediction = linear.predict(...)
```

Expected result. prediction has shape (120,). The table gives test MAE and RMSE in ms for both models, and the printed slopes retain ms/MB and ms per queued request units.

If you get stuck. Do not compare raw-unit slopes from linear.coef_ with scaled beta_next as if they had identical units. Do not refit using test labels.

Read the results before drawing conclusions

The gradient check concerns one optimization step on training data. The validation/test tables concern prediction on different rows. A lower RMSE than the mean baseline is evidence on this simulation, not a guarantee that every error is that small. The code uses np.testing/assert for numerical checks and display(pd.DataFrame(...)) only to format results.

The notebook provides the data-generation and split cells before task 1, and any additional task setup immediately before its starter. An `assert` stops execution when a condition is false. A passing check verifies that condition, not every scientific claim. `print` writes values as text; `display` renders tables. Read row and column labels before comparing numbers.

Written interpretation

Answer each question in two to four sentences. Support claims with the displayed values, a calculation, or a concrete counterexample.

1. Interpret the two fitted slopes, explicitly stating what is held fixed and the units.
2. Explain why nonzero test error remains even when the fitted family contains the simulator's mean relationship.
3. If the objective changes from half MSE to MSE, how do the gradient and an equivalent learning rate change?

Submission checklist

1. Complete all three task cells; leave supplied checks and data unchanged.
2. Restart the kernel and run every cell in order. Save the outputs, including any expected warnings, so another reader can follow the evidence.
3. Include the requested shapes, scores, probabilities, or parameter count and numbered written answers. Label units and distinguish validation from test.
4. State what was fixed in advance and what validation selected. Describe a limitation; do not change your choices after seeing test results.

Criterion	Points	Evidence
Implementation	4	Three tasks completed; checks pass and requested outputs exist.
Experiment	2	Training-only fitting; prescribed split and selection rule.
Interpretation	3	Reasoning tied to results, units or probabilities, and limitations.
Reproducibility	1	Fresh run, saved outputs, declared choices and warnings.

Check your understanding. Identify one failure that a single high accuracy or low loss would miss. Describe an additional check that could reveal it.